

COURSE CONTENT SUMMARY

TOPICS

READING

Python For Everybody (PYE) - Severance
Python's Object Oriented Programming (POO) - Phillips

Week 1	- course intro - Administrative - Install Python/Jupyter - Set up Git - Version control (Git Intro) - Python1 Intro: • Data types • conditionals • Functions	- PYE1: Why you should learn to write programs? - PYE2: Variables, expressions, and statements - PYE3: Conditional Execution - PYE4: Functions
Week 2	- Python Intro (cont.) • Strings • Iterations	- PYE5: Iteration - PYE6: Strings
Week 3	- Python Intro (cont.) • Lists • Tuples • Files • Lambda Functions • Dictionaries	- PYE7: Files - PYE8: Lists - PYE9: Dictionaries - PYE10: Tuples
Week 4	• Python classes • More on class method, Iterators, and Generators, Exceptions	- POO1: Object-oriented Design - POO2: Objects in Python - POO3: When Objects Are Alike SKIPPED
Week 5	• Algorithmic Thinking (Big-O notation) • Testing & Debugging	- Ch. 61-63 of Intro to Algorithms SKIPPED - Unit Test - Logging Facility in Python - Google Python Style Guide
Week 6	• Numerical computing (Numpy) • More on Numpy	Numpy (Package documentation)
Week 7	SciPy MIDTERM	SciPy (Package Documentation)

MIDTERM REVIEW

Week 1	- course intro - Administrative - Install Python/Jupyter - Set up Git - Version control (Git Intro) - Python1 Intro: • Data types • conditionals • Functions	- PYE1: Why you should learn to write programs? - PYE2: Variables, expressions, and statements - PYE3: Conditional Execution - PYE4: Functions
--------	---	--

1-1: Syllabus, Installing Python (and Jupyter), Intro to Git

- Python is a **(1) dynamically typed**, **(2) interpreted** programming language
 - (1): does not require you to declare a variable's type
 - (2): write code → Run directly w/ interpreter
- Git**: de facto standard for distributed version control system (VCS)
- VCS**: tools used to track changes to source code (or other collection of files/folders)

1-2: Data Types, Conditionals, Functions

- Error types
 - syntax, semantic, runtime (and logical)
- Built-in data types
 - Scalar (cannot be subdivided): int, float, bool, NoneType
 - Non-scalar: lists, strings, dictionaries
- Boolean expressions, logical operators, and conditionals
- **<Function, non-argument>** takes zero/more arguments and returns a value

Week 2	- Python Intro (cont.) • Strings • Iterations	- PYE5: Iteration - PYE6: Strings
--------	--	---

2-1: Strings, Iterations, and Lists

- Sequences: immutable sequence of case-sensitive characters
- + operations (concatenation (+), repeat (*)), Slicing (start: stop: [excluding: step]), comparisons (spaces/punctuation, digits, approach, names/symbols, lowercase)
- + immutability, string methods (str.upper(), str.find(), etc.), in keyword
- Iteration: run the same bit of code repeatedly
 - + while loop: (1) set loop var. on outside of while loop (2) test loop var. in cond'n (3) do something (4) modify loop var. with cond'n loop
 - must manually terminate or use break keyword
 - use continue keyword to skip rest of iteration and finishing loop body for come iteration
 - + for loop: run over elements of some set
 - useful when looping through a known sequence of items
 - + Recursion: using a function that calls itself; requires: (1) End condition (2) Function Body
 - + Recursion Error: type of Runtime Error when a recursive f. exceeds max depth of recursion (3) Function calls itself often via decreasing index
- Lists: sequences whose values can be any data type
- + creating (empty) lists, operations (+ and *), indexing, slicing, Mutability

Week 3	Python intro (cont.) Lists Tuples Files Lambda Functions Dictionaries	PYE7: Files PYE8: Lists PYE9: Dictionaries PYE10: Tuples
---------------	--	---

3-1: Lists, Tuples, Files

- Mutating lists (e.g. add, delete, readme (sort), etc.)
- + List operations (L: append, insert, extend, remove, pop); mutate original list but return None
- + Reordering (L.sort(), L.reverse(), sorted(L) - original list unchanged)
- + String to list (casting w/ list(), s.split('char'))
- + List to string (''.join(L))
- List comprehensions
 - [[x**2 for x in my_list if boolean_expr]]
- Tuples: immutable sequences whose values can be any data type
 - + creating, indexing & slicing, immutability, swap var. types
 - + Built-in function return, function input, zip() application → returns an iterator of tuples
- Files: ways to store and manage data on a computer
- + Reading Files: with open('file') as f, f.readline
 - open(filename, mode) were mode = { "r" - text mode, read-only; "w" - text mode, write; "a" - text mode, append }
 - unset for: read, write, append
 - File type: text or binary
- + Writing Files: open(mode='w'), f.write, f.close
 - creates new if file already exists
- + Pickle module: (de)serializing Python objects
 - Serialization: convert Python object(s) into byte stream (pickle.dump (Pyto-Bits))
 - Deserialization: convert byte stream back into Python obj (pickle.load (Byte-to-Py))
- + OS module: lets us interact w/ OS (os.getcwd, os.listdir)
 - Path: starts at root directory '/' and describes sequence of nested directories (absolute/relative)
- + Formatted strings: (1) % operator (old-fashioned - avoid) (2) String formatting (3) f-strings (newest - suggested)

3-2: Lambda Functions, Dictionaries

- Lambda Functions: creates a procedure/function object but does not bind a name to it
 - + useful when we don't want to name functions - especially simple ones
 - < lambda param: body_of_code...func_name >
- Dictionaries: has entries that map a key-value
 - + key: customised index, any immutable object
 - + value: can be any object
 - + creating, operations, iterate over dict, dict as counter, memoization application
 - + some computation for future use

Week 4	Python classes More on class method, iterators, and generators, exceptions	POOL: Object-Oriented Design POO2: Objects in Python POO3: When objects are alike	SKIPPED
---------------	---	---	----------------

4-1: Python classes (oop)

- creating and using a class: Define class name, class data attributes, and procedural attributes
 - Data (class) attributes: directly inside class but outside of any method
 - + __init__: (1) named: recommended method to initialize instance attributes
 - + self: allows you to create variables that belong to the object
 - always first parameter of method
 - parameter to refer to an instance of-class also creating one get
 - don't need to provide arguments for self; done automatically
 - + str - method: defines string representation of an object
 - should return a string when { print obj } { str(obj) }
 - procedural attributes: use self as first param., other parameters to the method
 - access class (data) attributes using dot notation

- Getters and Setters: used outside of class to access data attributes, e.g. `get-X()` and `set-X()`

- Nesting Objects: objects can have other objects as their attributes; call this an embedded attribute object

- objects are mutable

- Pure Functions & Modifiers: Pure Function - returns an object; does not modify any of its arguments
 + often easier to debug; common in functional programming
Modifier - changes attributes of one or more of its arguments
 + often faster & more efficient; common in OOP

note: method is like a function, but it is provided by an object
 → must include self as its first argument

- Inheritance: allows us to create new classes from old ones

Parent class: base/super class <classdef classname>

Child class: derived/sub-class <classdef classname (parentclass)>

- + inherit all data/behaviors of parent class
- + add more info/data and/or behavior; make or override behavior

4-2: More class methods, Iterators and Generators, Exceptions

- Additional Note about `__init__()` for child classes

- super() - `__init__()` to ensure the attributes in parent class properly initialized
 + followed by additional initialization steps specific to subclasses

- Iterators and Generators: Iterable: an object can be looped over if it has the `__iter__()` method

e.g. lists and all sequences (strings, tuples, etc.)
 + can loop over these objects

Iterator: object that represents a "data stream"; supports `__next__()` method

- + returns next element of stream/sequence
- + raises "StopIteration" when there are no elements left
- + `iter()` function creates Iterator object

Generator Expressions: create an iterator rather than list (w/ list comprehension)

- + more conducive for large/infinite sequences
- + use parentheses rather than brackets: `< (x/len(x) for x in my-list if boolean-expr) >`

Generators: work like functions, but they maintain internal state and yield (instead of return)

- + simple way to construct useful iterators (rather than creating a new class)

- Exception Handling and Assertions

try:
 # potentially problematic code

- instead of crash immediately, handle exceptions:

→ can handle specific except clauses to deal w/ a particular exception/error type

- Assertions: defensive programming tool

`assert` statement that should evaluate to true, "statement not true" =>

- + assert statement raises an AssertionError if assumptions not met

except:
 # handle problem
else:
 # always executed when try block completes w/ no exceptions
finally:
 # always executed (and runs before function returns) >

Week 5 {

- Algorithmic Thinking (Big-O notation)
- Testing & Debugging

{ Ch. 11-13 of Intro to Algorithms **SKIPPED**
 Unit Test
 Logging Facility in Python
 Google Python Style Guide

5.1: Algorithmic Thinking

- Timing programs and counting operations: i) start clock ii) call function iii) stop clock

clock time vs. clock perf-counter → designed for performance measurement

- counting operations

- Asymptotic growth: order of growth $\text{Big-O} = f(n) = O(g(n)) \Rightarrow \exists \text{ constants } c, n_0 \text{ for which}$

(upper bound) $c \cdot g(n) \geq f(n) \forall n > n_0$

$\text{Big-}\Omega = f(n) = \Omega(g(n)) \Rightarrow \exists \text{ constants } c, n_0 \text{ for which}$
 (lower bound) $f(n) \geq c \cdot g(n) \forall n > n_0$

$\text{Big-}\Theta = f(n) = \Theta(g(n)) \Rightarrow \exists \text{ constants } c_1, c_2, n_0 \text{ for which}$
 $c_2 \cdot g(n) \geq f(n) \geq c_1 \cdot g(n) \forall n > n_0$

- Finding $\Theta(n)$: Drop constants and multiplicative factors; focus in dominant terms:

$\Theta(1)$: constant runtime (independent of input size, e.g. `def >= 0: return`)

$\Theta(\log(n))$: e.g. binary search (each loop halves search interval)

$\Theta(n)$: linear running time (e.g. `def sum(x):`)

$\Theta(n \log(n))$: log-linear; scale well → balanced merge algorithms

(e.g. $\Theta(\log(n))$ divide and $\Theta(n)$ merge)

$\Theta(n^2)$: polynomial growth

$\Theta(n^3)$: exponential running time; sometimes replaced by faster alg's w/ memoization

(e.g. Fibonacci)

↓ $\Theta(n!)$: factorial runtime; for n distinct objects, there are $n!$ permutations

complexity

5.2: Test and Debugging

- Defensive Programming: Testing & Debugging

→ unit test, continuous integration test, regression test

- Week 6 {
- Numerical computing (Numpy)
 - More on Numpy
- Numpy (Package documentation)

6-1: Intro to Numpy

Numpy: open-sourced add-on modules for numerical computing

- Numpy as numerical computing (Basics)

- preliminary data types (e.g. bool, int32, float64)
- **np.ndarray**: N-dimensional array object; grid of values all have type
 - using **np.array**: 2D-functions (np.zeros, np.arange, np.linspace)
 - using **2D-functions** (np.eye, np.diag)
 - Properties of existing array (np.full-like, np.broadcast)

- Array indexing

- slicing, integer array indexing, boolean array indexing
- boolean operators (np.all, np.any, np.logical_and, np.logical_or)

- Math, vector, and Matrix operations

- **elementary array with operators** elementwise as (1) operator overload (2) functions in Numpy
- **array axis**: functions apply operations along specified axis
 - axis=0 (operates vertically, down through the rows)
 - axis=1 (operates horizontally, across columns)
- **vector operations** (np.dot)

- **Broadcasting**: allows Numpy to work w/ arrays of different shapes when

- performing arithmetic operations; for two arrays, compatible when (1) equal size (2) one of them is size 1
- behaves like it were copied along dimension

6-2: More on Numpy

- More on Broadcasting

- **np.newaxis** and **axis** used to add a new dimension to existing array (np.newaxis: np.newaxis; None)

- More on Matrix/Vector operations

- **'*' is elementwise multiplication** for np.ndarray
- **'*' is matrix multiplication** for np.matrix
 - instead use @ operator for np.matrix: user positive

- **Array stacking and splitting**: Array stacking: joining arrays along a new (or existing) axis (np.vstack, np.hstack, np.concatenate)

- **Universal functions (ufuncs)**: efficient, vectorized operations on array data (e.g. np.sin)

- **Numpy statistics**: implement all standard statistical distributions, functions

- **np.random**: contains methods for generating random numbers (np.random.choice, np.random.shuffle, np.random.permutation)

- **Stats**: dealt w/ generating built-in functions to deal w/ (np.random)

- **SciPy**: distinct package part of Numpy ecosystem
 - Provides statistical functions (e.g. stats tests)

Practice Problems

(a) (3 points) What will be the output of the following code?

```
numbers = [5, 10, 15, 20, 10]
numbers.append(25) [5, 10, 15, 20, 10, 25]
numbers.remove(10) [5, 10, 15, 20, 25]
result = numbers.pop() [5, 15, 20, 25]
print(numbers)
```

mutate original list

- ☒ [5, 15, 20, 10]
- ☐ [5, 15, 20]
- ☐ [10, 15, 20, 10]
- ☐ [5, 10, 15, 20]

(b) (3 points) Which of the following correctly creates a tuple containing only the integer 1?

- ☐ t = (1)
- ☒ t = (1,)
- ☐ t = tuple("1")
- ☐ t = tuple([1])

2. Please answer these questions with one or two complete sentences.

(a) (4 points) In Python, can you change the type of a variable after it's been declared? If so, demonstrate with a short code snippet.

Yes, Python is dynamically typed; don't have to declare a variable's type.

list((1,2,3)) > changes tuple: (1,2,3) into list: [1,2,3]

(b) (4 points) What is the purpose of the __init__ method in Python classes?

__init__ method initializes instance attributes.

Both class attributes are directly inside class but outside of any method.

Self allows you to create variables that belong to the object.

3. For each of the following code snippets, identify what type of error will occur (2 points) and explain why (1 point).

(a) (3 points) List, Dictionary and String Manipulations 1.

```
1 student_scores = {"Alice": 95, "Bob": 87, "Charlie": 92}
2 del student_scores["David"]
3 print(student_scores)
```

KeyError, "David" is not a recognized key in student_scores dict.

(a) (3 points) Error Handling 1

```
1 try:
2     y = 10 / 0
3 except ZeroDivisionError:
4     print("Division by zero!")
5 else:
6     print("else block")
7 finally:
8     print("finally block")
```

4. For each question, write the expected output of the given code snippet. If there's an error, explain what kind of error would occur.

Division by zero!
finally block

5. Time Complexity Analysis. For each of the following code snippets, determine the time complexity in Big O notation.

(a) (4 points) Nested Loop Analysis.

```
1 def nested_operations(n):
2     count = 0
3     for i in range(n): # n operations
4         for j in range(n): # n operations
5             count += 1
6             if count % 2 == 0:
7                 for k in range(10): # Fixed iteration of 10
8                     count += 1
9     return count
```

$\Rightarrow \Theta(n^2)$; dropping constants and multiplicative factors

WEEK 7 { SciPy
MIDTERM { SciPy (Package Documentation)

SciPy as Scientific Python

- Designed and used for scientific/technical computing
- + library (collection) of algorithms and mathematical tools built to work w/ NumPy arrays
- + everything in the namespaces of SciPy submodules is public

SciPy Linear Algebra

- + `scipy.linalg` contains all the functions in `numpy.linalg`
- could be faster

SciPy Sparse

- + Sparse matrix classes: CSC, CSR, etc.
- `scipy.sparse` module to identify and build sparse matrices

More (SciPy) Submodules

- + Statistics, signal, IO, etc. → loading/saving files
- + bioinformatics (e.g. motif finding)
- + HT's (real-time linear alg.)

WEEK 8 { Fall study break
Intro to ML and SciML-learn

- More on SciPy

- + `scipy.stats`, descriptive statistics, statistics models

- crash course on learning

- + AI, ML, deep learning

- Supervised Learning

- + define a (learned) mapping from input to output based on paired input/output data examples
- e.g. (univariate/multivariate) regression, (binary/multi-class) classification, image segmentation, depth estimation, pose estimation

- More difficult tasks

- + translation, image caption, image generation from text

- Unsupervised Learning

- + learning about a dataset w/o labels
- e.g. clustering, (probabilistic) generative models

- Reinforcement Learning

- + agent takes actions (decisions/movings); changes state to receive rewards from the environment
- e.g. chess

Week 9

Scikit-learn Matplotlib and Pandas

9-1: Scikit-learn in practice

- Classification ex. in Scikit-learn
 - recognizing handwritten digits using `SVM`
 - + fitting & predicting model, choosing different kernels
 - evaluating model performance
 - + Precision, Recall (FPS), Accuracy
- Regression ex. in Scikit-learn
 - `OLS` vs `LASSO regression`
 - penalty term for non-sparse coefficients
- Unsupervised learning in Scikit-learn
 - K-Gaussian Mixture Model (GMM)
 - + estimate each cluster via expectation maximization (EM)
- Model selection via information criterion
 - e.g. Bayesian/Akaike IC, Minimum CP, etc.
- Cross-validation; helps choose parameters
 - e.g. k-fold CV
- Assessing models (metrics): `coef`, `R2`, `Root Score`, etc.
- Model persistence: pickling model objects

9-2: Matplotlib and Pandas

Matplotlib: comprehensive library for creating static, animated, and interactive visualizations (similar to R's `ggplot2`)

- Basic plotting: `c`: import matplotlib.pyplot as plt
`plt.plot(x,y)`
- Customizing Plots:
 - line type/color, marker type; Multiple lines (in single plot)
 - titles/axis labels, legends, annotating figures, axis (nonlinear) range
 - `histogram`, `plt.hist()`, Bar plot, Box & whisker plot, Pie chart, Subplots
 - Saving images: `plt.savefig(filename)`, Animations

Pandas: open-source, modern data science tool; database-like structures (similar to those in R)

- Tidy Data
- Basic Data Structures:
 - `SERIES`: 1-d labeled (indexed array)
 - `DataFrame`: table of rows w/ labeled cols
- Pandas Series Basics
 - Built on top of NumPy arrays
 - contain custom index and optional name
 - can create from array-like structures (e.g. Python `list`)
 - Properties: `values`, `index`, `name`, `dtype`
- Pandas Series operations
 - indexing works like basic Python; supports slicing
 - series objects support most NumPy functions that act on arrays
 - + support unions so long as all their entries support operations
- Pandas DataFrames
 - Fundamental unit of pandas; 2-d structure (rows/columns); analogous to R `df`
 - can be created from many different objects (e.g. `dict`, `Series`, `ndarray`)
 - rows/cols as attributes, column manipulation, indexing/selection: `df.loc('row')`
`df.loc(row, column)`
 - Arithmetic: Series aligned to DataFrames via row-wise broadcasting, removing NAs: `dropna` method
 - Reading/Writing Files (e.g. `pd.read_csv()`), summarizing DataFrames: `head`, `info`, `describe` methods
 - Statistical operations: methods that operate along `rows` w/ operate on cols (axis=0)
cols w/ operate on rows (axis=1)
 - Row- and Column-Wise Functions: `df.apply(func, axis, args)`

Week 10

Pandas & Seaborn Intro to Database w/ SQL

10-1: Pandas and Seaborn

- Reorganizing Data: Group By
 - splitting data based on some criteria, applying functions to different splits, combining results into a single data structure
 - `groupby` method returns a pandas `groupby` object
 - + contains groups attribute: `dict` that maps group labels to indices
 - Aggregation: summary stats for each (specified) column
 - + hierarchically-nested aggregation
 - `groupby.get_group` lets us pick out an individual group
 - `groupby` objects also support `aggregate` method: often more convenient
 - + supply a list of functions to apply column-wise
 - + `dict` whose keys: columns, values: Functions
- Statistical and Other Computations
 - `pd.change_period`: periods arg: specifies time lag
 - `corr` method: compute covariance b/w a series and another series
 - `rank` method: returns a new series whose values are the data ranks

- Iterating over Series and DataFrame
 - + often slow; instead, use more efficient vectorized operations or apply/reduce operation
- Filtering data: filter method
- combining DataFrames: concat method
- Stacking and Pivoting
 - c.d.f. pivot(index, columns, values) -> to unstack
 - stack method returns stacked df

- Visualize: Seaborn

- library for making statistical graphs
- built on top of matplotlib; integrates closely w/ pandas data structures



10-2: Intro to Database with SQL

- Resurgence of Deep Learning

- From spreadsheet (pandas) to Database

- Database: collection of data organized for creating, reading, updating, and deleting
- Relational Database: collection of tables (structured data)
- Database Management System (DBMS): software with which you can interact w/ a database

- What is SQL

SQL: structured Query Language; create, read, update, and delete data in a database

< SELECT [column name] FROM [table] >

< SELECT [column name] FROM [table] WHERE [filter] >

- supports many (or) natural comparisons one would want

< SELECT DISTINCT [column] FROM [table] >

- remove repeats from a set of returned results

< GROUP BY field: x > combines the rows w/ the same value in the field: x; supports many operations: { SUM, AVG, MIN, MAX, COUNT }

AS (keyword): gives nicer name to the aggregated field

HAVING (keyword): operates after applying filters and GROUP BY

< SELECT [column] FROM [table] ORDER BY [column] [ASC|DESC] >

- Relational DB's

Pros

- Natural for vast majority of applications
- Numerous tools for managing and querying

Cons

- Not user-tailored to some data (e.g. networks, unstructured text)
- Fixed schema (i.e. hard to add columns)
- Expensive to maintain when data gets large (e.g. many TB's of data)

- Spilkes module: provides SQL interface in Python compliant w/ the database



represents the connection to the on-disk database

In order to execute SQL statements and fetch results from SQL queries, we need to use a database cursor: < res = cur.execute(query) >
- to retrieve results, use the cur.fetchone and cur.fetchall methods

Week 11

- Intro to Deep Learning (PyTorch)
- Loss Function, (Stochastic) GD

11-1: Intro to Deep Learning & Pytorch

- Intro to Pytorch

Python-based open-sourced scientific computing package targeted at two sets of audiences:

- replacement for NumPy to use the power of GPU's
- to deep learning research platform that provided maximum flexibility and speed

- Basic Datatypes: tensor: generalization of vectors and matrices
- + multi-dimensional matrix containing elements of a single data type
- + Properties: shape, shape, device - can be supported on GPU

- creating tensors

- + w/ built-in functions; based on existing tensors

- operation examples: add, reshape

- tensor softmax a NumPy Array

- torch.device ("cuda") -> tensor on GPU

- + compute united device Architecture: NVIDIA's parallel computing platform and programming model that allows NVIDIA's GPU for general-purpose computing

- + GPU speedup on Google cloud

- Automatic Differentiation & Gradients

- + requires_grad - True tells Pytorch to track all operations performed on tensor to compute gradients (derivatives) w/ respect to it later

- essential for auto-differentiation

- tells Pytorch how to compute derivatives when going 'backward' through an operation during backpropagation

- when we call backward(), Pytorch uses these grad_fns to compute

- alternatively use torch.autograd.grad() directly

- Perceptron and MLP

- Perceptron: single neuron; structural building block of deep learning
- Forward propagation: $\hat{y} = g(w \cdot x + \frac{1}{2} \sum_{i=1}^n w_i x_i) = g(z)$
- Activation functions: help introduce non-linearity, e.g. $\begin{cases} \text{sigmoid function} \\ \text{hyperbolic tangent} \\ \text{rectified linear unit (ReLU)} \end{cases}$
- Multi-output Perceptron: $\hat{z} = w_0 + \sum_{i=1}^n w_i x_i$; $\hat{y} = w_0 + \sum_{i=1}^n w_i x_i$
- + one simplified perceptron: $\hat{z} = w_0 + \sum_{i=1}^n w_i x_i$
- Shallow (single-layer) Neural Network
- Deep (multi-layer) Neural Network

11: Loss Function, (Stochastic) Gradient Descent

- The Loss Function

A measure of how "correct" (how bad) the model is getting $\begin{cases} \text{Predicted: } F(x^i; w) \\ \text{Actual: } y^i \end{cases}$

Empirical Loss: $J(w) = \frac{1}{n} \sum_{i=1}^n L(F(x^i; w), y^i)$

Different learning problem has different measures and thus have different loss constructions: regression, classification, customized (for scope of this class), e.g.

Mean Square Error (MSE) Loss: $J(w) = \frac{1}{n} \sum_{i=1}^n (y_i - F(x^i; w))^2$

Loss for regression problem

Binary Cross Entropy Loss: $J(w) = -\frac{1}{n} \sum_{i=1}^n y^i \log(F(x^i; w)) + (1 - y^i) \log(1 - F(x^i; w))$

Loss for (binary) classification

- Training a Model

$w^* = \underset{w}{\operatorname{argmin}} J(w)$ via:

Stochastic GD	Mini-Batch GD
1) Initialize weights randomly $\sim N(0, \sigma^2)$	1) Initialize weights randomly $\sim N(0, \sigma^2)$
2) Loop until convergence:	2) Loop until convergence:
a) Pick single data point	a) Pick batch of B data points
b) compute gradient: $\partial L / \partial w$	b) compute gradient (across batches)
• $\partial L / \partial w$ can be computationally expensive	$\partial L / \partial w = \frac{1}{B} \sum_{i=1}^B \partial L(w) / \partial w$
c) update weights: $w \leftarrow w - \eta (\partial L / \partial w)$	c) update weights: $w \leftarrow w - \eta (\partial L / \partial w)$
3) Return weights	3) Return weights

Backpropagation: $\textcircled{x} \xrightarrow{w} \textcircled{y} \xrightarrow{w_2} \textcircled{z} \rightarrow J(w)$

computing w_2 : $\frac{\partial J(w)}{\partial w_2} = \frac{\partial J(w)}{\partial y} \frac{\partial y}{\partial w_2}$; repeat this for every weight in the network using later layers

Recall: $w \leftarrow w - \eta \frac{\partial L}{\partial w}$ where η = learning rate

- small η converges slowly and gets stuck in false local minima
- large η overshoots, becomes unstable, and diverges

$\begin{cases} \text{trial-and-error} \\ \text{adaptive learning rate} \end{cases}$

Strategy: stable learning rate converges smoothly and avoids local minima

Q: How? $\begin{cases} 1) \text{ try lots of different learning rates \& see what works} \\ 2) \text{ better: design an adaptive learning rate that "adapts"} \\ \text{to the landscape} \end{cases}$

Gradient Descent algorithms: AdaDelta, AdaFactor, Adagrad, Adam (w), Rprop, SGD

Week 12: Stochastic Gradient Descent, DataLoader, Sequential modeling

Deep Sequential Modeling (RNN & LSTM)

12-1: Stochastic Gradient Descent, DataLoader, Sequential modeling

- Stochastic w/ mini-batches

Gradient Descent: $w^* = \underset{w}{\operatorname{argmin}} \frac{1}{n} \sum_{i=1}^n L(F(x^i; w), y^i)$

using gradient descent to update weights via: $\frac{\partial L}{\partial w}$ where $w \leftarrow w - \eta \frac{\partial L}{\partial w}$

Full batch: $\frac{\partial L}{\partial w} = \frac{1}{n} \sum_{i=1}^n \frac{\partial L(w)}{\partial w}$ $\begin{cases} \text{Batch size} = n; 1 \text{ update per epoch} \\ \text{use all data} \\ \text{very slow; doesn't work w/ stale} \\ \text{can get stuck @ local minima} \end{cases}$

Mini-batch: $\frac{\partial L}{\partial w} = \frac{1}{B} \sum_{i=1}^B \frac{\partial L(w)}{\partial w}$ $\begin{cases} \text{use subdatasets; choose smaller batch size} \\ \text{w/ 100 updates per epoch} \\ \text{more accurate estimate of gradient} \\ \text{smoother convergence} \\ \text{allow for larger learning rates} \end{cases}$

Stochastic GD: $\frac{\partial L}{\partial w}$ where $\begin{cases} \text{Batch size} = 1; n \text{ updates per epoch} \\ \text{use single sample} \\ \text{most efficient} \\ \text{noisy updates, unstable} \end{cases}$

- Regularization

Problem of overfitting

underfitting: model does not have capacity to fully learn the data

ideal fit

↓ overfitting: performs well on training data - but poorly on test data's
often too complex, does not generalize well

To reduce overfitting

Regularization: technique that constrains our optimization problem to discourage complex models;
to improve generalization of our model

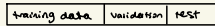
Dropout: During training, randomly set some activations to 0

- + originally drop prob: 0.5 (0.2, ~0.5, etc.); can use different dropout prob's in different layers
- + Force network not to rely on every node

why it works: network will learn not to rely on particular connections too heavily

- will consider more connections (b/c it cannot rely on individual ones)
- weight values will be more spread-out (may lead to smaller weights like L2 norm)

Early Stopping: 1) split dataset into 3 parts: training, validation, test data:



- use validation accuracy for tuning
- use test set once at the end (for unbiased estimate of generalization performance)

2) Early Stopping

- reduce overfitting by observing the training/validation accuracy gap during training and stopping at the "right" point



Add more data: if not possible, data augmentation can be helpful
(e.g. for images: random rotation, crop, translation)

12-2: Deep Sequential Modeling (RNN, LSTM)

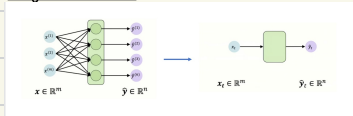
Sequence Data

Given an image of a ball, predict where it will go next
e.g.: Audio, Stock Market, Language, etc.

Sequence Modeling applications

- one-to-one: Binary classification
- many-to-one: Sentiment Analysis
- one-to-many: Image captioning
- many-to-many: Machine Translation

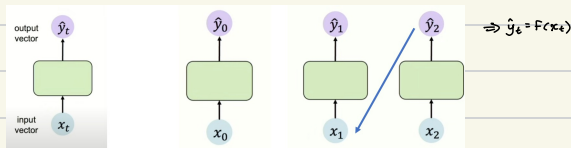
Adding the notion of time



Handling individual time steps

apply the same model sequence to time slice

NOTE: won't work if there is no link between time steps



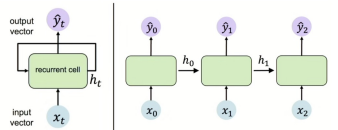
Neurons w/ Recurrence

- Need to build a NN that can explicitly model the time step to time step relation: neuron w/ recurrence, i.e.

$$\hat{y}_t = F(x_t, h_{t-1})$$

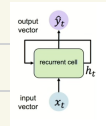
⋮
⋮
⋮

output
input
past memory



Recurrent Neural Networks (RNN)

• add a cell state: cause we apply a recurrence relation at every time step

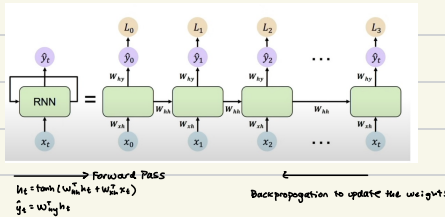


to process sequence data: $h_t = f_w(x_t, h_{t-1})$, e.g. $h_t = \tanh(W_{sh}^T h_{t-1} + W_{sx}^T x_t) \Rightarrow \hat{y}_t = W_{hy} h_t$

hidden state function weights input previous hidden state activation function

note! in RNN, we use the same function and the set of parameters at every time step

computational graph



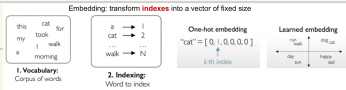
RNNs: design criteria

Handle variable-length sequences

- track long term dependencies
- maintain information about order
- share parameters across the sequence

Ex: predict the next word, encoding language for NN

Representing the word (encoding language)



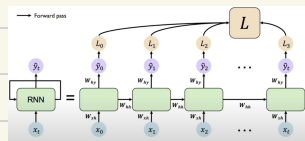
Ex: capture differences in order, e.g.

- the food was good, not bad at all
- the food was bad, not good at all

Loss Function

• in an RNN, defined based on the loss at every time step

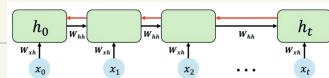
$$J(W) = \frac{1}{n} \sum_{t=1}^n L(f(x_t; W), y_t) = \frac{1}{n} \sum_{t=1}^n L(\hat{y}_t, y_t)$$



Backpropagation through time

• Backpropagation is done at each point in time;
 at timestep T , the derivative of the loss w.r.t. weight matrix W as expressed by

$$\frac{\partial L}{\partial W} = \sum_{t=1}^T \frac{\partial L}{\partial W} \frac{\partial L}{\partial W} \frac{\partial L}{\partial W}$$



• Problem w/ gradients

- difficult to capture long-term dependencies b/c of multiplicative gradients that can be exponentially decreasing/increasing w.r.t. to the number of layers

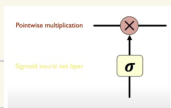
vanishing gradient: many values < 1
 + activation function
 + weight initialization
 + network architecture

exploding gradients: many values > 1
 - gradient clipping
 - some big gradients

• if we multiply many small numbers together, we would have smaller/smaller gradient, and the bias parameters are only there to capture short-term dependencies

• to resolve, use **gates** to selectively add/remove info with each recurrent unit:

gated cell: $\left\{ \begin{array}{l} \text{Gated Recurrent Unit (GRU)} \\ \text{Long Short-Term Memory Units (LSTM)} \end{array} \right\}$ can optionally let info through the cell



$$f = \sigma(W_f x^{(t)} + U_f x^{(t-1)} + b_f)$$

• specific gates are used and each has a well-defined purpose:

Type of Gate	Role	Used in
update gate Γ_u	How much past should matter now	GRU, LSTM
reset gate Γ_r	Does previous information	GRU, LSTM
forget gate Γ_f	Erase a cell or not	LSTM
output gate Γ_o	How much to reveal of a cell	LSTM

LSTM

• use gates to control the flow of information (forget, store, update, and output)

Characterization	Gated Recurrent Unit (GRU)	Long Short-Term Memory (LSTM)
$\tilde{c}^{(t)}$	$\tanh(W_c [\Gamma_r * a^{(t-1)}, x^{(t-1)}] + b_c)$	$\tanh(W_c [\Gamma_r * a^{(t-1)}, x^{(t-1)}] + b_c)$
$c^{(t)}$	$\Gamma_u * \tilde{c}^{(t)} + (1 - \Gamma_u) * c^{(t-1)}$	$\Gamma_u * \tilde{c}^{(t)} + \Gamma_f * c^{(t-1)}$
$a^{(t)}$	$\tilde{c}^{(t)}$	$\Gamma_o * c^{(t)}$
Dependencies		

Week 13: $\left\{ \begin{array}{l} \text{Deep Sequential Modeling (Attention and Transformers)} \end{array} \right\}$

Week 14: $\left\{ \begin{array}{l} \text{CNN in PyTorch} \end{array} \right\}$

Week 18: $\left\{ \begin{array}{l} \text{Deep Generative Modeling and VAE} \end{array} \right\}$

Week 19: Deep Generative Modeling